

NUMN33: Project Part 3

Arpita Bhattacharya and Carlotta Boi: Homework Group 6

March 21, 2025

Task 1. Consider the parabolic equation

$$\frac{\partial}{\partial t} u = \Delta u - u \quad \text{in } (0, 1) \times [0, T] \quad (0.0.1)$$

$$\frac{\partial}{\partial x} u(0, t) = \frac{\partial}{\partial x} u(1, t) = 0 \quad \text{and } u(\cdot, 0) = \eta \quad (0.0.2)$$

1. Formulate the implicit Euler/FEM discretization of equation 0.0.1 with an equidistant grid in space, i.e., $h_i = h$ for all i .

2. Derive the linear equation system that needs to be solved in every time step. Make sure that you explicitly state the entries of the mass and stiffness matrices.

1. We first write 0.0.1 in weak form, therefore we multiply by a test function v and we integrate over the spatial domain:

$$\int_0^1 \frac{\partial u}{\partial t} v dx = \int_0^1 \Delta u v dx - \int_0^1 u v dx$$

Integrating by parts and since the boundary term vanishes because of Neumann condition, we obtain:

$$\int_0^1 \frac{\partial u}{\partial t} v dx + \int_0^1 \frac{\partial}{\partial x} u \frac{\partial}{\partial x} v dx + \int_0^1 u v dx = 0.$$

Therefore the weak formulation is: find u such that for all test function $v \in V$

$$\left(\frac{\partial}{\partial t} u, v \right) + a(u, v) + (u, v) = 0,$$

with

$$(u, v) = \int_0^1 u v dx$$

and

$$a(u, v) = \int_0^1 \frac{\partial}{\partial x} u \frac{\partial}{\partial x} v dx.$$

We first do a discretization in space. We use a finite element method on an equidistant grid. Let $x_i = ih$, $i = 0, 1, \dots, N$ and $h = \frac{1}{N}$. We choose the finite element space $V_h \subset V$ to be the space of continuous, piecewise linear function on the partition $\{x_i\}$.

$$V_h = \{u \in C(\bar{\Omega}) : v|_{[x_i, x_{i+1}]} \in \mathbb{P}_1\}.$$

where

$$\mathbb{P}_1 = \{p: \mathbb{R} \rightarrow \mathbb{R} : p(x) = a^T x + b, a \in \mathbb{R}, b \in \mathbb{R}\}.$$

Let $\{\phi_i\}_{i=0}^N$ be the hat functions, which is a basis for V_h . The finite element approximation is:

$$u_h(x) = \sum_{j=0}^N u_j \phi_j(x).$$

We then do a discretization in time, using the implicit Euler method. Let $t^n = n\Delta t$, with $n = 0, 1, 2, \dots, M$ and we approximate $u(x, t^n)$ by $u^n(x)$. The time derivative is approximated as:

$$\frac{\partial}{\partial t} u \approx \frac{u_h^{n+1} - u_h^n}{\Delta t}.$$

And the the finite element approximation becomes:

$$u_h^n(x) = \sum_{j=0}^N u_j^n \phi_j(x).$$

Therefore the weak formulation becomes:

$$\left(\frac{u_h^{n+1} - u_h^n}{\Delta t}, \phi_i \right) + a(u_h^{n+1}, \phi_i) + (u_h^{n+1}, \phi_i) = 0,$$

for each test function ϕ_i . Where

$$\left(\frac{u_h^{n+1} - u_h^n}{\Delta t}, \phi_i \right) = \frac{1}{\Delta t} \left(\sum_{j=0}^N (u_j^{n+1} - u_j^n) \phi_j, \phi_i \right) = \frac{1}{\Delta t} \sum_{j=0}^N (u_j^{n+1} - u_j^n) \int_0^1 \phi_j(x) \phi_i(x) dx.$$

The mass matrix is defined as:

$$M_{i,j} = \int_0^1 \phi_i(x) \phi_j(x) dx,$$

so we have:

$$\frac{1}{\Delta t} \sum_{j=0}^N M_{i,j} (u_j^{n+1} - u_j^n).$$

The stiffness term is:

$$a(u_h^{n+1}, \phi_i) = \int_0^1 (u_h^{n+1})'(x) \phi_i'(x) dx = \sum_{j=0}^N u_j^{n+1} \int_0^1 \phi_j'(x) \phi_i'(x) dx.$$

The stiffness matrix is defined as:

$$A_{i,j} = \int_0^1 \phi_i'(x) \phi_j'(x) dx,$$

so we have:

$$\sum_{j=0}^N A_{i,j} u_j^{n+1}.$$

The reaction term is:

$$(u_h^{n+1}, \phi_i) = \int_0^1 u_h^{n+1}(x) \phi_i(x) dx = \sum_{j=0}^N u_j^{n+1} \int_0^1 \phi_j(x) \phi_i(x) dx,$$

thus

$$\sum_{j=0}^N M_{i,j} u_j^{n+1}.$$

Assembling we have:

$$\frac{1}{\Delta t} \sum_{j=0}^N M_{i,j} (u_j^{n+1} - u_j^n) + \sum_{j=0}^N A_{i,j} u_j^{n+1} + \sum_{j=0}^N M_{i,j} u_j^{n+1} = 0.$$

We want to write it in matrix form, therefore let $u^{n+1} = (u_0^{n+1}, \dots, u_N^{n+1})^T$, $u^n = (u_0^n, \dots, u_N^n)^T$ such that:

$$\frac{1}{\Delta t} M(u^{n+1} - u^n) + Au^{n+1} + Mu^{n+1} = 0,$$

that we can rewrite as:

$$u^{n+1} (M + \Delta t(A + M)) = u^n (M).$$

Finally the initial condition $u(x, 0) = \eta(x)$ is imposed by setting $u^0 = (u_0^0, \dots, u_N^0)^T$ as the vector of nodal interpolation of η , $u_i^0 = \eta(x_i)$.

2.

Consider:

$$\left(\frac{u_h^{n+1} - u_h^n}{\Delta t}, \phi_i \right) + a(u_h^{n+1}, \phi_i) + (u_h^{n+1}, \phi_i) = 0.$$

Where $\{\phi_i\}_{i=0}^N$ are the hat functions that form a basis, they are defined on a uniform grid $0 = x_0 < x_1 < \dots < x_N = 1$, with $h = x_i - x_{i-1} = x_{i+1} - x_i$. Therefore, we have:

$$\phi_i(x) = \begin{cases} \frac{x - x_{i-1}}{h}, & x \in [x_{i-1}, x_i] \\ \frac{x_{i+1} - x}{h}, & x \in [x_i, x_{i+1}] \\ 0, & \text{otherwise.} \end{cases}$$

We want $\phi_i(x) = 1$ at $x = x_i$, $\phi_i(x) = 0$ at $x = x_{i+1}$, $\phi_{i+1}(x) = 0$ at $x = x_i$ and $\phi_{i+1} = 1$ at $x = x_{i+1}$ and linear on each subinterval.

We want to compute the local mass matrix on $[x_i, x_{i+1}]$,

$$\int_{x_i}^{x_{i+1}} \phi_i(x) \phi_j(x) dx, \quad j \in \{i, i+1\}.$$

Therefore we have to compute:

$$\begin{aligned} & \int_{x_i}^{x_{i+1}} \phi_i(x) \phi_i(x) dx, \\ & \int_{x_i}^{x_{i+1}} \phi_{i+1}(x) \phi_{i+1}(x) dx, \\ & \int_{x_i}^{x_{i+1}} \phi_i(x) \phi_{i+1}(x) dx. \end{aligned}$$

We first compute

$$\int_{x_i}^{x_{i+1}} \phi_i(x) \phi_i(x) dx = \int_{x_i}^{x_{i+1}} \phi_i^2(x) dx,$$

let $s = x - x_i$, thus we have

$$\phi_i(x) = \frac{x_{i+1} - x}{h} = \frac{x_{i+1} - (s + x_i)}{h} = \frac{h - s}{h}.$$

Since when $x = x_i$ then $s = 0$ and when $x = x_{i+1} = x_i + h$ then $s = h$, the integral becomes:

$$\begin{aligned} \int_{x_i}^{x_{i+1}} \phi_i^2(x) dx &= \int_0^h \phi_i^2(s) ds = \int_0^h \left(\frac{h-s}{h}\right)^2 ds = \\ \frac{1}{h^2} \int_0^h (h-s)^2 ds &= \frac{1}{h^2} \int_0^h (h^2 + s^2 - 2hs) ds = \frac{1}{h^2} [h^2 s - hs^2 + \frac{s^3}{3}]_0^h = \frac{h}{3}. \end{aligned}$$

Now we compute the following:

$$\int_{x_i}^{x_{i+1}} \phi_i(x) \phi_{i+1}(x) dx,$$

where $\phi_{i+1} = \frac{x - x_i}{h} = \frac{s}{h}$. Therefore

$$\int_{x_i}^{x_{i+1}} \phi_i(x) \phi_{i+1}(x) dx = \int_0^h \frac{s(h-s)}{h^2} ds = \frac{1}{h^2} \int_0^h (hs - s^2) ds = \frac{1}{h^2} [\frac{hs^2}{2} - \frac{s^3}{3}]_0^h = \frac{h}{6}.$$

Therefore the local mass matrix is:

$$M^i = \begin{pmatrix} \int_{x_i}^{x_{i+1}} \phi_i^2 & \int_{x_i}^{x_{i+1}} \phi_i \phi_{i+1} \\ \int_{x_i}^{x_{i+1}} \phi_{i+1} \phi_i & \int_{x_i}^{x_{i+1}} \phi_{i+1}^2 \end{pmatrix}.$$

Which is

$$M^i = \begin{pmatrix} \frac{h}{3} & \frac{h}{6} \\ \frac{h}{6} & \frac{h}{3} \end{pmatrix}.$$

When we assemble into a global matrix, the interior nodes will be: $M_{i,i} = \frac{h}{3} + \frac{h}{3} = \frac{2h}{3}$, with $i = 1, \dots, N-1$. While $M_{i,i+1} = M_{i+1,i} = \frac{h}{6}$, for $i = 0, \dots, N-1$. Whereas $M_{0,0} = M_{N,N} = \frac{h}{3}$. Consequently the global mass matrix form is tridiagonal:

$$M = \begin{pmatrix} \frac{h}{3} & \frac{h}{6} & 0 & \dots & \dots & 0 \\ \frac{h}{6} & \frac{2h}{3} & \frac{h}{6} & 0 & \dots & 0 \\ 0 & \frac{h}{6} & \frac{2h}{3} & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \frac{h}{6} \\ 0 & 0 & \dots & \dots & \frac{h}{6} & \frac{h}{3} \end{pmatrix}.$$

The local stiffness matrix is given by:

$$A_{i,j} = \int_0^1 \phi_i'(x) \phi_j'(x) dx.$$

On $[x_i, x_{i+1}]$ we have that $\phi_i(x) = \frac{x_{i+1}-x}{h}$, therefore $\phi_i'(x) = -\frac{1}{h}$, whereas $\phi_{i+1}(x) = \frac{x-x_i}{h}$, thus $\phi_{i+1}'(x) = \frac{1}{h}$. Then we have:

$$\int_{x_i}^{x_{i+1}} \phi_i'^2(x) dx = \int_{x_i}^{x_{i+1}} \left(-\frac{1}{h}\right)^2 dx = \frac{1}{h^2} \int_{x_i}^{x_{i+1}} dx = \frac{1}{h^2} h = \frac{1}{h}.$$

For an interior node we have $A_{i,i} = \frac{1}{h} + \frac{1}{h} = \frac{2}{h}$, with $i = 1, \dots, N-1$. While, for boundary nodes $A_{0,0} = A_{N,N} = \frac{1}{h}$. On $[x_i, x_{i+1}]$, for $A_{i,i+1}$ we have:

$$\int_{x_i}^{x_{i+1}} \phi'_i(x) \phi'_{i+1}(x) dx = \int_{x_i}^{x_{i+1}} -\frac{1}{h} \frac{1}{h} dx = -\frac{1}{h}.$$

Consequently we have: $A_{i,i+1} = A_{i+1,i} = -\frac{1}{h}$, for $i = 0, \dots, N-1$. Assembling into a global matrix, the global stiffness matrix is:

$$A = \begin{pmatrix} \frac{1}{h} & -\frac{1}{h} & 0 & \dots & \dots & 0 \\ -\frac{1}{h} & \frac{2}{h} & -\frac{1}{h} & 0 & \dots & 0 \\ 0 & -\frac{1}{h} & \frac{2}{h} & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & -\frac{1}{h} \\ 0 & 0 & \dots & \dots & -\frac{1}{h} & \frac{1}{h} \end{pmatrix}.$$

At each time step the discrete system is:

$$(M + \Delta t(A + M))u^{n+1} = Mu^n.$$

i.e.

$$\sum_{j=0}^N (M_{i,j} + \Delta t(A_{i,j} + M_{i,j}))u_j^{n+1} = \sum_{j=0}^N M_{i,j}u_j^n, \text{ with } i = 0, \dots, N.$$

Denoting $B = (M + \Delta t(A + M))$, for $i = 0$, we have:

$$B_{0,0} = M_{0,0} + \Delta t(A_{0,0} + M_{0,0}) = \frac{h}{3} + \Delta t\left(\frac{1}{h} + \frac{h}{3}\right)$$

$$B_{0,1} = M_{0,1} + \Delta t(A_{0,1} + M_{0,1}) = \frac{h}{6} + \Delta t\left(-\frac{1}{h} + \frac{h}{6}\right)$$

the right hand side is:

$$(Mu^n)_0 = M_{0,0}u_0^n + M_{0,1}u_1^n = \frac{h}{3}u_0^n + \frac{h}{6}u_1^n$$

together:

$$\left(\frac{h}{3} + \Delta t\left(\frac{1}{h} + \frac{h}{3}\right)\right)u_0^{n+1} + \left(\frac{h}{6} + \Delta t\left(-\frac{1}{h} + \frac{h}{6}\right)\right)u_1^{n+1} = \frac{h}{3}u_0^n + \frac{h}{6}u_1^n.$$

For the interior nodes $i = 1, \dots, N-1$:

the diagonal entries are:

$$B_{i,i} = M_{i,i} + \Delta t(A_{i,i} + M_{i,i}) = \frac{2}{3}h + \Delta t\left(\frac{2}{h} + \frac{2}{3}h\right),$$

while the off - diagonal entries are:

$$B_{i,i-1} = B_{i,i+1} = \frac{h}{6} + \Delta t\left(-\frac{1}{h} + \frac{h}{6}\right),$$

the right hand side is:

$$(Mu^n)_i = \frac{h}{6}u_{i-1}^n + \frac{2}{3}hu_i^n + \frac{h}{6}u_{i+1}^n.$$

Together:

$$\left(\frac{h}{6} + \Delta t\left(-\frac{1}{h} + \frac{h}{6}\right)\right)u_{i-1}^{n+1} + \left(\frac{2}{3}h + \Delta t\left(\frac{2}{h} + \frac{2}{3}h\right)\right)u_i^{n+1} + \left(\frac{h}{6} + \Delta t\left(-\frac{1}{h} + \frac{h}{6}\right)\right)u_{i+1}^{n+1} = \frac{h}{6}u_{i-1}^n + \frac{2}{3}hu_i^n + \frac{h}{6}u_{i+1}^n$$

Finally, for $i = N$

$$B_{N,N} = M_{N,N} + \Delta t(A_{N,N} + M_{N,N}) = \frac{h}{3} + \Delta t(\frac{1}{h} + \frac{h}{3}),$$

$$B_{N,N-1} = \frac{h}{6} + \Delta t(-\frac{1}{h} + \frac{h}{6}),$$

the right hand side is:

$$(Mu^n)_N = M_{N,N-1}u_{N-1}^n + M_{N,N}u_N^n = \frac{h}{6}u_{N-1}^n + \frac{h}{3}u_N^n,$$

together

$$(\frac{h}{6} + \Delta t(-\frac{1}{h} + \frac{h}{6}))u_{N-1}^{n+1} + (\frac{h}{3} + \Delta t(\frac{1}{h} + \frac{h}{3}))u_N^{n+1} = \frac{h}{6}u_{N-1}^n + \frac{h}{3}u_N^n.$$

Task 2. Prove that the Crank-Nicolson/FEM scheme converges as $O(h^2 + (\frac{t}{n})^2)$ under the assumptions stated in Lecture 12 (10), i.e., prove the theorem on page 6, Lecture 12 (10).

Consider the following operators:

$A : D(A) \rightarrow H$, maximal dissipative on H ; unbounded

$A_h : H_h \rightarrow H_h$, maximal dissipative on H_h ; bounded but $\|A_h\|_{H_h} \rightarrow \infty$ as $h \rightarrow 0$

$P_h : H \rightarrow H_h$, the orthogonal projection of H onto H_h

$Q_h := A_h^{-1}P_hA$ and $Z_h := (A_h^{-1}P_h)^2A^2 = A_h^{-2}P_hA^2$.

Consider the assumptions:

Assumption. (E). The spaces $\{(H_h, (\cdot, \cdot)_H)\}$ are chosen such that

$$\|A^{-1}f - A_h^{-1}P_hf\|_H \leq \text{const } h^2\|f\|_H, \forall f \in H.$$

Assumption. (C). (Maximal dissipativity)

$$\begin{aligned} (Au, u)_H &\leq 0, & \forall u \in D(A), \\ R(I - hA) &= H & \forall h > 0. \end{aligned}$$

In particular, the operators A and A_h are maximal dissipative.

Consider the following lemma and theorem:

Lemma 0.0.3. If (C) and (E) are valid and $\eta \in D(A^2)$, $\eta_h \in H_h$, then

$$\|e^{tA} - e^{tA_h}\eta_h\|_H \leq \|\eta - \eta_h\|_H + \text{const}(t) \cdot h^2(\|A\eta\|_H + \|A^2\eta\|_H),$$

Theorem 0.0.4. If the operator $A : D(A) \rightarrow H$ is maximal dissipative and $\eta \in D(A^3)$, then

$$\|(I + \frac{t}{2n}A)(I - \frac{t}{2n}A)^{-1})^n - e^{tA}\eta\|_H \leq \text{Const}(t)(\frac{t}{n})^2\|A^3\eta\|_H.$$

We have to prove the following theorem:

Theorem 0.0.5. If assumption (C) and (E) are valid and $\eta \in D(A^3)$, $\eta_h \in H_h$, then

$$\|e^{tA}\eta - [(I + \frac{t}{2n}A_h)(I - \frac{t}{2n}A_h)^{-1}]^n\eta_h\|_H \leq \|\eta - \eta_h\|_H + C(t)(h^2 + (\frac{t}{n})^2) \sum_{k=1}^3 \|A^k\eta\|_H$$

To prove it, let $S_k := (A_h^{-1}P_h)^k A^k$, $Z_h = S_3$, note that $(I - S_3)\eta = \sum_{k=1}^3 (s_{i-1} - s_i)\eta = O(h^2)$, and let $\gamma_h = [(I + \frac{t}{2n}A_h)(I - \frac{t}{2n}A_h)^{-1}]^n$. We can write:

$$e^{tA}\eta - \gamma_h\eta_h = e^{tA}\eta - e^{tA_h}S_3\eta + (e^{tA_h} - \gamma_h)S_3\eta + (\gamma_h)(S_3\eta - \eta_h).$$

If we choose $\eta_h := S_3\eta$ in lemma 0.0.3, and notice that $\eta \in D(A^3)$ we have

$$\|e^{tA}\eta - e^{tA_h}S_3\eta\|_H \leq \|(I - S_3)\eta\|_H + C(t)h^2(\|A\eta\|_H + \|A^2\eta\|_H).$$

Theorem 0.0.4 yields:

$$\begin{aligned} \|(e^{tA_h} - \gamma_h)S_3\eta\|_H &\leq C(t) \frac{t^2}{n^2} \|A_h^3 S_3\eta\|_H \\ &= C(t) \frac{t^2}{n^2} \|A_h^3 (A_h^{-1}P_h)^3 A^3\eta\|_H \\ &= C(t) \frac{t^2}{n^2} \|P_h A^3\eta\|_H \leq C(t) \frac{t^2}{n^2} \|A^3\eta\|_H \end{aligned}$$

Furthermore, as A_h is maximal dissipative $\|(y_h)\|_{H_h} \leq 1$ and

$$\|(\gamma_h)(S_3\eta - \eta_h)\|_H \leq \|(I - S_3)\eta\|_H + \|\eta - \eta_h\|_H.$$

It remains to bound $(I - S_3)\eta$:

$$\begin{aligned} (I - S_3)\eta &= (I - Q_h)\eta + (Q_h - S_3)\eta \\ &= (A^{-1} - A_h^{-1}P_h)A\eta + (A_h^{-1}P_hA - (A_h^{-1}P_h)^3A^3)\eta \\ &= (A^{-1} - A_h^{-1}P_h)A\eta + A_h^{-1}P_h(A^{-2} - (A_h^{-1}P_h)^2)A^3\eta \end{aligned}$$

Then using (E):

$$\|(I - S_3)\eta\|_H \leq Ch^2(\|A\eta\|_H + \|A^3\eta\|_H).$$

Task 4.

$$-\nabla \cdot (K(u)\nabla u) = f$$

$f=1$ is a forcing term and the diffusion tensor is

$$K(u) = (\varepsilon^2 + |\nabla u|^2)^{\frac{p-2}{2}}$$

with ε being a small positive number. On the boundary we prescribe Dirichlet conditions $g=0$. We solve this problem in variational form

$$\int_{\Omega} K(u)\nabla u \cdot \nabla v \, dx = \int_{\Omega} f v \, dx$$

on a domain $\Omega = [0, 1]^2$.

First we set up the grid for the domain $\Omega = [0, 1]^2$ and space a linear Lagrange space V_h .

```
domain = cartesianDomain([0,0],[1,1],[5,5])
gridView = leafGrid(domain)
space = lagrange(gridView)
```

Next, we set up u and v as the trial and test function. Then, we define the gradients of these functions, so they can be used in the weak form equations.

Along with it, we define $K(u)$ and set up the constants ε, p, f and g .

```
# Define trial and test functions
u = TrialFunction(space)
v = TestFunction(space)

# Define constants
epsilon = Constant(1e-5)
p = Constant(3.0)
f = Constant(1.0)
g = Constant(0.0)

# Define the diffusion coefficient K(u)
K = lambda u: (epsilon**2 + inner(grad(u), grad(u))**((p-2)/2))

# Weak formulation
a = inner(K(u) * grad(u), grad(v)) * dx
L = f * v * dx
```

Finally, we apply the Dirichlet boundary conditions and then we create a galerkin scheme, solve the system and store the result in u_h .

```
# Apply Dirichlet boundary conditions
dbc = DirichletBC(space, g)
scheme = galerkin([a==L, dbc], solver="cg")

uh = space.function(name="solution")

# Solve the variational problem
scheme.solve(target=uh)

uh.plot()
```

We obtain the plot result as shown in Fig.0.1.

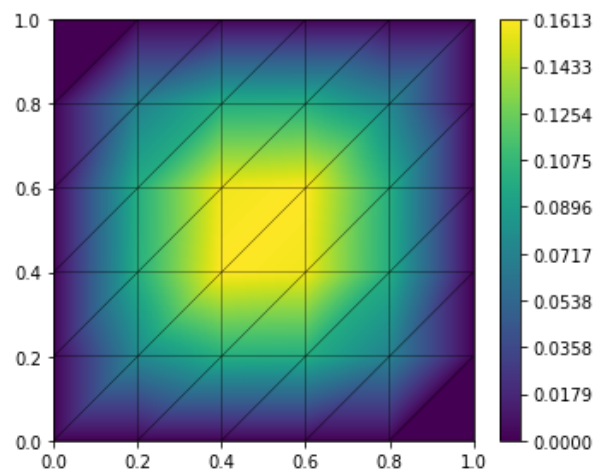


Figure 0.1: Plot

Task 5.

$$\partial_t u - \nabla \cdot K(\nabla u) \nabla v = f$$

$f = f(x, t)$ is a time dependent forcing term and the diffusion tensor is

$$K(u) = \frac{2}{1 + \sqrt{1 + 4|\nabla u|}}$$

On the boundary we prescribe Neumann boundary conditions and require initial conditions $u(\cdot, 0) = u_0(\cdot)$. We solve this problem in variational form using Implicit Euler in time.

$$\int_{\Omega} \frac{u^{n+1} - u^n}{\Delta t} v + K(u^{n+1}) \nabla u^{n+1} \cdot \nabla v dx = \int_{\Omega} f(x, t^n + \Delta t) v dx + \int_{\partial\Omega} g(x, t^n + \Delta t) \cdot n v ds \quad (0.0.6)$$

on a domain $\Omega = [0, 1]^2$. We choose f, g so that the exact solution is:

$$u_{ex}(x, t) = e^{-2t} \left(\frac{1}{2}(x^2 + y^2) - \frac{1}{3}(x^3 - y^3) \right) + 1.$$

Define constants

```
from dune.ufl import Constant
dt = Constant(0, name="dt") # time step
t = Constant(0, name="t") # current time
```

First we define the domain

```
domain = cartesianDomain([0, 0], [1, 1], [4, 6])
gridView = leafGrid(domain)
```

We define the space and the discrete functions to store the solution at the current time step and the next time step.

```
space = solutionSpace(gridView, order=2)

# current solution and previous solution
u_h = space.function(name='u_h')
u_h_n = u_h.copy(name="previous")
```

Then we define the spatial coordinate and the initial condition:

```
# spatial coordinate and initial condition
x = SpatialCoordinate(space)
initial = 1/2*(x[0]**2 + x[1]**2) - 1/3*(x[0]**3 - x[1]**3) + 1
```

Finally we define the time step and the current time step as constants:

```
dt = Constant(0, name="dt") # time step
t = Constant(0, name="t") # current time
```

Task 6. Implement the weak form a and right hand side b given in 0.0.6 using terms K, f , and g . Note: Both f and g can be implemented in terms of u_{ex} and UFL, e.g.

```
f = lambda s: dtExact(t) - div( K(exact(s))*grad(exact(s)) )
g = lambda s: K(exact(s))*grad(exact(s))
```

We define the trial and test functions, u and v respectively.

```
u = TrialFunction(space)
v = TestFunction(space)
```

After that, we define $K(u)$

```
abs_du = lambda u: sqrt(inner(grad(u), grad(u)))
K = lambda u: 2/(1 + sqrt(1 + 4*abs_du(u)))
```

Then we define the exact solution:

```
exact = lambda s: exp(-2*s)*(initial - 1) + 1
```

and the time derivative of the exact solution

```
dt_exact = lambda s: -2*exp(-2*s)*(initial - 1)
```

We define the forcing f and the boundary flux g

```
f = lambda s: dt_exact(s) - div(K(exact(s))*grad(exact(s)))
g = lambda s: K(exact(s))*grad(exact(s))
```

Then we define weak form a and the right hand side b :

```
a = ( dot((u - u_h_n)/dt, v) + dot(K(u)*grad(u), grad(v)) ) * dx
n = FacetNormal(space)
b = f(t+dt)*v*dx + dot(g(t+dt), n)*v*ds
```

Task 7. Solve 0.0.6 for $T = 0.25$. This requires a time loop and the choice of an appropriate time step size Δt . Start with something small, e.g. $\Delta t = \frac{T}{100}$ or even $\Delta t = \frac{T}{1000}$. Note: u^n enters the weak form as a parameter which needs to be updated.

```
while time <= endTime:
    u_h_n.assign(u_h)
    scheme.solve(target=u_h)
    time += scheme.model.dt # same name on model as the above constant dt
    scheme.model.t = time # same name on model as the above constant t
```

We setup a solution scheme:

```
scheme = solutionScheme(a == b, solver='cg')
```

Then we define the parameters, we interpolate the initial condition into u_h , after that we define the time loop. We then compute the errors against the exact solution at T .

```
endTime = 0.25
dt.value = endTime/100
time = 0
u_h.interpolate(initial)
while time <= endTime:
    u_h_n.assign(u_h)
    scheme.solve(target=u_h)
    time += dt.value
    scheme.model.t = time
exact_end = exact(endTime)
```

```
l2error = gridFunction(dot(u_h - exact_end, u_h - exact_end), name="l2error")
h1error = gridFunction(dot(grad(u_h - exact_end), grad(u_h - exact_end)), name="
h1error")
errors = [np.sqrt(e) for e in integrate([l2error, h1error])]
```

We obtain for $\Delta t = \frac{T}{100}$:

$$|u_h - u_{ex}| = 3.66455e - 04$$

$$|\nabla(u_h - u_{ex})| = 3.098136e - 03.$$

We can refine the grid twice and recompute the solution on the finer grid and we can compute L^2 -error and H^1 -error, and we will have a second order convergence for the L^2 norm and second order convergence for the H^1 norm. If we use $\Delta t = \frac{T}{500}$, we will see that H^1 order is approximately 2 while L^2 is above 2. If instead, we use $\Delta t = \frac{T}{1000}$, we will have that H^1 order is approximately 2, instead for L^2 we have order 3.

Task 8. Implement a Crank-Nicolson time scheme for 0.0.6.

The Crank Nicolson method for the time discretization is:

$$\int_{\Omega} \frac{u^{n+1} - u^n}{\Delta t} v + \frac{1}{2} K(\nabla u^{n+1}) \nabla u^{n+1} \nabla v + \frac{1}{2} K(\nabla u^n) \nabla u^n \nabla v dx$$

$$= \int_{\Omega} \frac{1}{2} (f(x, t^n) + f(x, t^n + \Delta t)) v dx - \int_{\partial\Omega} \frac{1}{2} (g(x, t^n) + g(x, t^n + \Delta t)) \cdot n v ds$$

We proceed as before, but we change a and b as:

```
a = ( dot((u - u_h_n)/dt, v) \
+ 0.5*dot(K(u)*grad(u), grad(v)) \
+ 0.5*dot(K(u_h_n)*grad(u_h_n), grad(v)) ) * dx
b = 0.5*(f(t)+f(t+dt))*v*dx + 0.5*dot(g(t)+g(t+dt),n)*v*ds
```

And for the time loop we write:

```
endTime = 0.25
exact_end = exact(endTime)
l2error = gridFunction(dot(u_h - exact_end, u_h - exact_end), name="l2error")
h1error = gridFunction(dot(grad(u_h - exact_end), grad(u_h - exact_end)), name="
h1error")
dt.value = 0.01
time = 0
u_h.interpolate(initial)
while time < (endTime - 1e-6):
    t.value = time
    u_h_n.assign(u_h)
    scheme.solve(target=u_h)
    time += dt.value
```

The values we obtain are the following:

$$|u_h - u_{ex}| = 1.12888e - 04$$

$$|\nabla(u_h - u_{ex})| = 3.06547e - 03$$

After we refine the grid twice, we have that the convergence is cubic for L^2 -error and is quadratic for H^1 -error.

Task 9. Consider the Cahn-Hilliard equation with $\psi : \mathbb{R} \rightarrow \mathbb{R}$ be defined as

$$\psi(x) = \frac{(1-x^2)^2}{4} \text{ and } \phi(x) := \psi(x)' = x^3 - x$$

The strong form for the solution $u: \Omega \times [0, T] \rightarrow \mathbb{R}$ is given by:

$$\begin{aligned} \partial_t u - \Delta(\phi(u) - \varepsilon^2 \Delta u) &= 0 \text{ in } \Omega \times [0, T] \\ u(\cdot, 0) &= u_0(\cdot) \text{ in } \Omega \\ \partial_n u &= \partial_n(\phi(u) - \varepsilon^2 \Delta u) = 0 \text{ on } \partial\Omega \times [0, T] \end{aligned}$$

The above equation contains 4th order derivatives which cannot be directly discretized with standard Finite Element methods, however, we can introduce an auxiliary variable

$$\eta = \phi(u) - \varepsilon^2 \Delta u$$

to transform the equation into a system of 2 equations only containing derivatives up to 2nd order. We obtain:

$$\partial_t u - \Delta \eta = 0 \quad \text{in } \Omega \times [0, T] \quad (0.0.7)$$

$$\eta - \phi(u) + \varepsilon^2 \Delta u = 0 \quad \text{in } \Omega \times [0, T] \quad (0.0.8)$$

$$u(\cdot, 0) = u_0(\cdot) \quad \text{in } \Omega \quad (0.0.9)$$

$$\eta(\cdot, 0) = \eta_0(\cdot) = \phi(u_0(\cdot)) - \varepsilon^2 \Delta u_0(\cdot) \quad \text{in } \Omega \quad (0.0.10)$$

$$\partial_n u = 0 \quad \text{on } \partial\Omega \times [0, T] \quad (0.0.11)$$

$$\partial_n \eta = 0 \quad \text{on } \partial\Omega \times [0, T] \quad (0.0.12)$$

with Neumann-Null boundary conditions (also called do-nothing boundary conditions in this case).

In order to understand the coupling we write this as a vector valued problem, with $\mathbf{u} := (u_0, u_1)^T$ (with $u_0 = u$ and $u_1 = \eta$ in the above equations). For the test functions from our discrete space we have $\mathbf{v} := (v_0, v_1)^T$. We derive a weak form for this equation in the same way as before by multiplying with a test function, integrating over the domain Ω and applying integration by parts to the terms with 2nd order derivatives. For the time derivative we use an implicit Euler discretization.

We obtain:

$$\begin{aligned} \int_{\Omega} \frac{u_0^{n+1} - u_0^n}{\tau} v_0 + \nabla u_1^{n+1} \cdot \nabla v_0 &= 0 \\ \int_{\Omega} u_1^{n+1} v_1 - \phi(u_0^{n+1}, u_0^n) v_1 - \varepsilon^2 \nabla u_0^{n+1} \cdot \nabla v_1 &= 0 \end{aligned}$$

Note that we also replaced $\phi(u_0^{n+1})$ with $\phi(u_0^{n+1}, u_0^n)$ to achieve energy stability. We follow the approximation by Eyre which reads $\phi(v, w) = v^3 - w$. We want to solve a vector valued problem and therefore create a Lagrange space over a vector valued range with dimension 2. Formulate the above described weak form.

```

# space for solution
space = lagrange(gridView, order=1, dimRange=2)

# discrete functions needed for form
u_prev = space.interpolate([0]*2, name="u_h_prev")
u_h = space.interpolate([0]*2, name="u_h")

u = TrialFunction(space)
v = TestFunction(space)

tau = Constant(1., name="tau") # timestep constant
eps = 0.05
eps2 = Constant(eps**2, name="eps2") # we need eps*eps

```

We begin with the set of equations that is the strong form of the Cahn-Hilliard equation with an auxiliary variable η . The function $\phi(u)$ is given by the Eyre's approximation $\phi(u^{n+1}, u^n) = (u^{n+1})^3 - u^n$.

For the first equation $\partial_t u - \Delta \eta = 0$, we can multiply by test function v_0 and integrate over Ω to get:

$$\int_{\Omega} (\partial_t u - \Delta \eta) v_0 dx = 0$$

Then we integrate it by parts to the Laplacian $\Delta \eta$ and use the Neumann-Null boundary conditions $\partial_n \eta = 0$.

$$\int_{\Omega} \partial_t u \cdot v_0 dx + \int_{\Omega} \nabla \eta \cdot \nabla v_0 dx = 0$$

Discretizing using the Implicit Euler method gives us the provided weak form:

$$\int_{\Omega} \frac{u_0^{n+1} - u_0^n}{\tau} v_0 dx + \int_{\Omega} \nabla u_1^{n+1} \cdot \nabla v_0 dx = 0$$

Next, for the equation $\eta - \phi(u) + \varepsilon^2 \Delta u = 0$, we can multiply by test function v_1 and integrate over Ω to get:

$$\int_{\Omega} (\eta - \phi(u) + \varepsilon^2 \Delta u) v_1 dx = 0$$

Then we integrate it by parts to the Laplacian Δu and use the Neumann-Null boundary conditions $\partial_n u = 0$.

$$\int_{\Omega} u_1 v_1 dx - \int_{\Omega} \phi(u) v_1 dx - \varepsilon^2 \int_{\Omega} \nabla u_0 \cdot \nabla v_1 dx = 0$$

The Eyre's approximation $\phi(u^{n+1}, u^n) = (u^{n+1})^3 - u^n$ gives us the provided weak form:

$$\int_{\Omega} u_1^{n+1} v_1 dx - \int_{\Omega} ((u^{n+1})^3 - u^n) v_1 dx - \varepsilon^2 \int_{\Omega} \nabla u_0^{n+1} \cdot \nabla v_1 dx = 0$$

In the Python code, we implement both the weak forms together as one equation. That is,

$$\int_{\Omega} \frac{u_0^{n+1} - u_0^n}{\tau} v_0 dx + \int_{\Omega} \nabla u_1^{n+1} \cdot \nabla v_0 + \int_{\Omega} u_1^{n+1} v_1 dx - \int_{\Omega} ((u^{n+1})^3 - u^n) v_1 dx - \varepsilon^2 \int_{\Omega} \nabla u_0^{n+1} \cdot \nabla v_1 dx = 0$$

After defining the grid and putting in the given snippet of the code, we formulate the weak form and solve the system:

```
# Define the weak form
weak_form = (
    inner((u[0] - u_prev[0])/tau, v[0]) * dx + inner(grad(u[1]), grad(v[0])) * dx
    + inner(u[1], v[1]) * dx - inner((u[0])**3 - u_prev[0], v[1]) * dx
    - eps2 * inner(grad(u[0]), grad(v[1])) * dx)

# Create the Galerkin scheme and solver
scheme = galerkin(weak_form == 0, solver="cg")

# Solve the system
scheme.solve(target=u_h)
```

Task 10. Solve the equations (0.0.7)-(0.0.12) with the following initial data:

```
from ufl import conditional, sin, pi
def initial(x):
    h = 0.01
    g0 = lambda x, x0, T:
        conditional(x-x0<-T/2, 0, conditional(x-x0>T/2, 0, sin(2*pi/T*(x-x0))**3))
    G = lambda x, y, x0, y0, T: g0(x, x0, T)*g0(y, y0, T)
    return 0.5*G(x[0], x[1], 0.5, 0.5, 50*h)
df = space.interpolate(initial, name="solution")
```

until $T = 2$ in $\Omega = [0, 1]^2 \subset \mathbb{R}^2$ with a decently fine grid, e.g. more than 5000 elements and $\tau = 1e-3$.

In the code we create a structured cartesian grid over the domain $\Omega = [0, 1]^2$ with 71×71 (about more than 5000) elements. Then we define a vector-valued Lagrange space:

```
domain = cartesianDomain([0,0],[1,1],[71,71])
gridView = leafGrid(domain)
space = lagrange(gridView, order=1, dimRange=2)
```

As we have a two dimensional vector $[u, \eta]$. We then use the provided snippet:

```
def initial(x):
    h = 0.01
    g0 = lambda x, x0, T: conditional(
        x - x0 < -T/Constant(2),
        0,
        conditional(
            x - x0 > T/Constant(2),
            0,
            sin(2*pi/T*(x - x0))**3
        )
    )
    G = lambda x, y, x0, y0, T: g0(x, x0, T) * g0(y, y0, T)
    return 0.5 * G(x[0], x[1], 0.5, 0.5, 50*h)
```

and after that we use it to define a vector valued function, where we set η equal zero. We then interpolate the initial data into the space.

And we define a time loop

```
T_final = 2.0
t = 0.0
while t < T_final:
    print("Solving at time t =", t)
    info = scheme_obj.solve(target=u_h)
    u_prev.assign(u_h)
    t += float(tau.value)
```

Where in each iteration, the non linear problem $F(u^{n+1}, v)$ is solved for the new time step and then the old solution is updated. Finally, we visualise the result through two plots as showing Fig 0.2 and Fig 0.3. In figure 0.2, we can see two yellow areas where u is high and two darker areas (dark blue) where u is low, therefore we have a phase separation.

If instead we use the implicit treatment of $\phi = u^3 - u$, then our weak form for the second equation changes. The implicit nonlinear term evaluated at time u^{n+1} becomes

$$\phi(u^{n+1}) = (u^{n+1})^3 - u^{n+1},$$

hence the code for the equation two becomes:

```
eq2 = (
    u[1] * v[1] * dx
    - ((u[0]**3) - u[0]) * v[1] * dx
    - eps2 * dot(grad(u[0]), grad(v[1])) * dx
)
```

We don't use $u_{prev}[0]$ anymore. And we initialize η as $\eta^0 = (u^0)^3 - u^0$, and therefore our code changes in this way:

```
def initial_vector(x):
    u0 = initial(x)
    eta0 = u0**3 - u0
    return (u0, eta0)
```

Now taking a grid over the domain $\Omega = [0, 1]^2$ with 21×21 elements, for the implicit treatment we will have figure 0.4 while for the Eyre treatment we have figure 0.5. From the figures, we can say that we have small variations of u in the implicit version, it seems that solution has only separated a little. While we have more variations for u in Eyre version. We tried to investigate further for the implicit version, and we noticed that if we change ε to a smaller value, like for instance 0.001 and the time step τ to 0.1, we have a mid variation, as can be seen in figure 0.6.

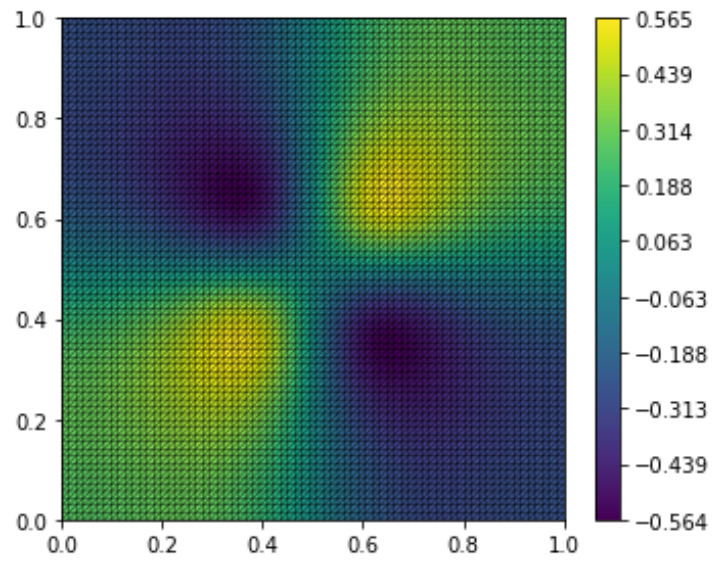


Figure 0.2: u component: $\phi = u^3 - v$ (Eyre)

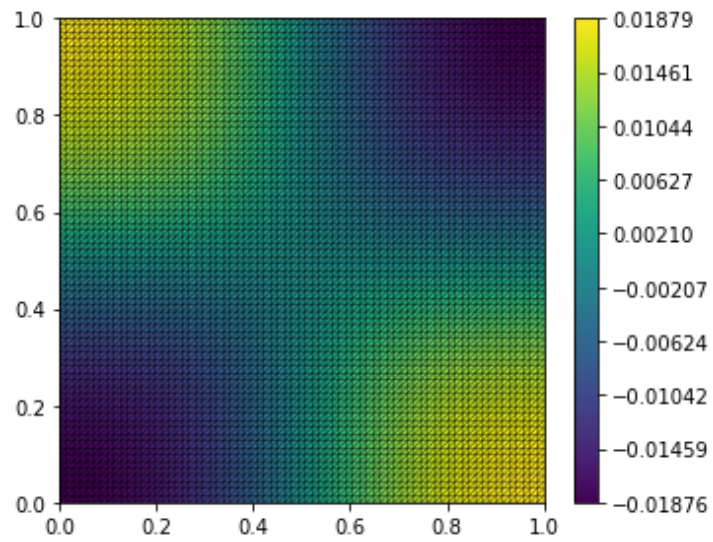


Figure 0.3: η component: $\phi = u^3 - v$ (Eyre)

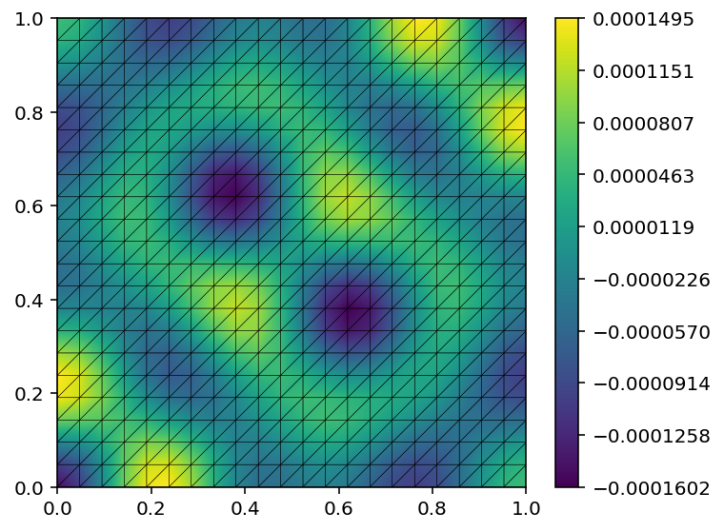


Figure 0.4: $\phi = \nu^3 - \nu$ (implicit)

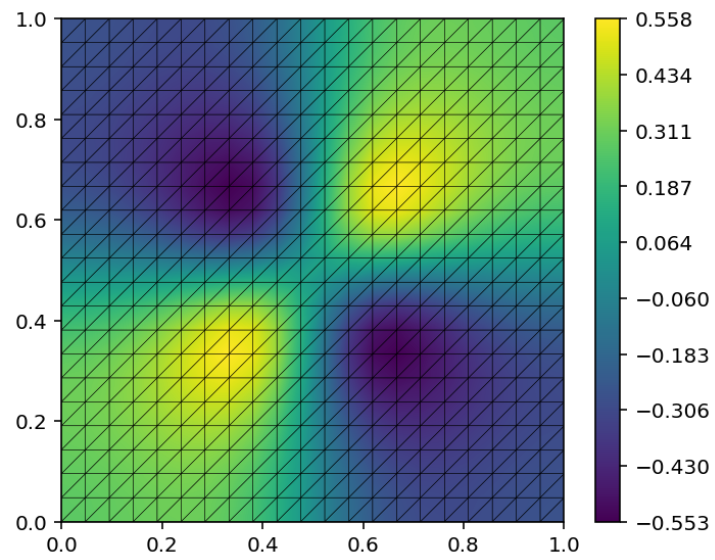
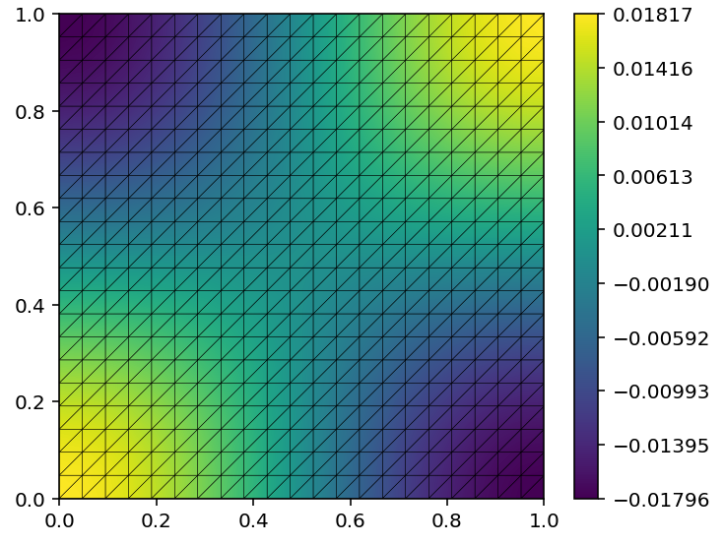


Figure 0.5: $\phi = u^3 - \nu$ (Eyre)

Figure 0.6: $\phi = \nu^3 - \nu$ (implicit)

.1 Appendix

Task 5-6-7

```
import dune.fem as fem
from dune.grid import cartesianDomain
from dune.alugrid import aluConformGrid as leafGrid
from dune.fem.scheme import galerkin as solutionScheme
from dune.fem.function import gridFunction
from dune.fem import integrate
import numpy as np
from ufl import (TestFunction, TrialFunction, SpatialCoordinate, FacetNormal,
                 dx, ds, grad, div, inner, dot, sqrt, exp)
from dune.ufl import Constant
from dune.fem.space import lagrange as solutionSpace

# create a domain and grid on [0,1]^2
domain = cartesianDomain([0, 0], [1, 1], [4, 6])
gridView = leafGrid(domain)
space = solutionSpace(gridView, order=2)

# current solution, previous solution
u_h = space.function(name='u_h')
u_h_n = u_h.copy(name="previous")

# spatial coordinate and initial condition
x = SpatialCoordinate(space)
initial = 1/2*(x[0]**2 + x[1]**2) - 1/3*(x[0]**3 - x[1]**3) + 1

# define time step and current time as constants
dt = Constant(0, name="dt") # time step
t = Constant(0, name="t") # current time
```

```

# define trial and test functions
u = TrialFunction(space) # corresponds to  $u^{(n+1)}$ 
v = TestFunction(space)

abs_du = lambda u: sqrt(inner(grad(u), grad(u)))

K = lambda u: 2/(1 + sqrt(1 + 4*abs_du(u)))

# define the exact solution u_ex(x,t)
exact = lambda s: exp(-2*s)*(initial - 1) + 1

# Time derivative of the exact solution
dt_exact = lambda s: -2*exp(-2*s)*(initial - 1)

f = lambda s: dt_exact(s) - div(K(exact(s))*grad(exact(s)))
g = lambda s: K(exact(s))*grad(exact(s))

# Define the boundary normal
n = FacetNormal(space)

# implicit Euler
# left-hand side
a = ( dot((u - u_h_n)/dt, v) + dot(K(u)*grad(u), grad(v)) ) * dx
# right-hand side
b = f(t+dt)*v*dx + dot(g(t+dt), n)*v*ds

# define the solution scheme with a conjugate gradient solver
scheme = solutionScheme(a == b, solver='cg')

endTime = 0.25
dt.value = endTime/100 # endTime/1000

time = 0

# interpolate the initial condition into u_h
u_h.interpolate(initial)

# time-stepping loop
while time <= endTime:
    # update previous solution
    u_h_n.assign(u_h)
    # solve the variational problem
    scheme.solve(target=u_h)

```

```

# update the time.
time += dt.value
scheme.model.t = time # update

# compute errors
exact_end = exact(endTime)
l2error = gridFunction(dot(u_h - exact_end, u_h - exact_end), name="l2error")
h1error = gridFunction(dot(grad(u_h - exact_end), grad(u_h - exact_end)), name="
    h1error")
errors = [np.sqrt(e) for e in integrate([l2error, h1error])]

print('grid size:', gridView.size(0))
print('\t | u_h - u_ex | =', '{:0.5e}'.format(errors[0]))
print('\t | nabla (u_h - u_ex) | =', '{:0.5e}'.format(errors[1]))

#visualize the solution
u_h.plot()
gridView.writeVTK('parabolic',
    pointdata={'u': u_h, 'l2error': l2error, 'h1error': h1error})

# global refinement of the grid
gridView.hierarchicalGrid.globalRefine(2)

# reduce the time step and update the scheme's dt parameter
dt.value /= 4
scheme.model.dt = dt.value # Ensure the scheme model uses the new dt

# reinitialize the solution and reset time
u_h.interpolate(initial)
time = 0

# time loop using scheme.model.dt and scheme.model.t
while time < (endTime - 1e-12):
    scheme.model.t = time # update model's time parameter
    u_h_n.assign(u_h)      # update previous solution
    scheme.solve(target=u_h) # solve for the new time level
    time += scheme.model.dt # increment time by the model's dt

# compute errors and output results
errorsFine = [np.sqrt(e) for e in integrate([l2error, h1error])]
print('grid size:', gridView.size(0))
print('\t | u_h - u | =', '{:0.5e}'.format(errorsFine[0]))
print('\t | grad(u_h - u) | =', '{:0.5e}'.format(errorsFine[1]))
u_h.plot()
eocs = [ round(np.log(fine/coarse)/np.log(0.5),2)
        for fine,coarse in zip(errorsFine,errors) ]
print("EOCs:",eocs)

```

Task 8

```

import dune.fem as fem
from dune.grid import cartesianDomain
from dune.alugrid import aluConformGrid as leafGridView
from dune.alugrid import aluConformGrid as leafGrid
from dune.fem.scheme import galerkin as solutionScheme
from dune.fem.function import gridFunction
from dune.fem import integrate

import numpy as np
from ufl import (TestFunction, TrialFunction, SpatialCoordinate, FacetNormal,
                 dx, ds, grad, div, inner, dot, sin, cos, pi, conditional)
from ufl import FacetNormal, ds, dx, inner, dot, div, grad, sqrt, exp,
                 TrialFunction, TestFunction
from dune.ufl import Constant
from dune.fem.space import lagrange as solutionSpace

domain=cartesianDomain([0,0], [1,1], [4,8])

gridView=leafGrid(domain)

space = solutionSpace(gridView, order=2)
u_h = space.function(name='u_h')
u_h_n = u_h.copy(name="previous")

x = SpatialCoordinate(space)
initial = 1/2*(x[0]**2+x[1]**2) - 1/3*(x[0]**3 - x[1]**3) + 1

dt = Constant(0, name="dt") # time step
t = Constant(0, name="t") # current time
u = TrialFunction(space)
v = TestFunction(space)

abs_du = lambda u: sqrt(inner(grad(u), grad(u)))
K = lambda u: 2/(1 + sqrt(1 + 4*abs_du(u)))
a = ( dot((u - u_h_n)/dt, v) \
      + 0.5*dot(K(u)*grad(u), grad(v)) \
      + 0.5*dot(K(u_h_n)*grad(u_h_n), grad(v)) ) * dx

exact = lambda t: exp(-2*t)*(initial - 1) + 1
f = lambda s: -2*exp(-2*s)*(initial - 1) - div( K(exact(s))*grad(exact(s)) )
g = lambda s: K(exact(s))*grad(exact(s))
n = FacetNormal(space)
b = 0.5*(f(t)+f(t+dt))*v*dx + 0.5*dot(g(t)+g(t+dt),n)*v*ds
scheme = solutionScheme(a == b, solver='cg')
endTime = 0.25
exact_end = exact(endTime)
l2error = gridFunction(dot(u_h - exact_end, u_h - exact_end), name="l2error")

```

```

hlerror = gridFunction(dot(grad(u_h - exact_end), grad(u_h - exact_end)), name="
    hlerror")
dt.value = 0.01
time = 0
u_h.interpolate(initial)
while time < (endTime - 1e-6):
    t.value = time
    u_h_n.assign(u_h)
    scheme.solve(target=u_h)
    time += dt.value

errors = [np.sqrt(e) for e in integrate([l2error,hlerror])]
print('grid size:', gridView.size(0))
print('\t | u_h - u | =', '{:0.5e}'.format(errors[0]))
print('\t | grad(uh - u) | =', '{:0.5e}'.format(errors[1]))
u_h.plot()
gridView.writeVTK('forchheimer', pointdata={'u': u_h, 'l2error': l2error, '
    hlerror': hlerror})

gridView.hierarchicalGrid.globalRefine(2)

dt.value /= 4
u_h.interpolate(initial)
time = 0
while time < (endTime - 1e-6):
    t.value = time
    u_h_n.assign(u_h)
    scheme.solve(target=u_h)
    time += dt.value

errorsFine = [np.sqrt(e) for e in integrate([l2error,hlerror])]
print('grid size:', gridView.size(0))
print('\t | u_h - u | =', '{:0.5e}'.format(errorsFine[0]))
print('\t | grad(uh - u) | =', '{:0.5e}'.format(errorsFine[1]))
u_h.plot()
eocs = [ round(np.log(fine/coarse)/np.log(0.5),2)
        for fine,coarse in zip(errorsFine,errors) ]
print("EOCs:",eocs)

```

Task 10 - Eyre

```

try:
    import dune.vem
except:
    print("This example needs 'dune.vem' - skipping")
    import sys
    sys.exit(0)

from scipy.sparse.linalg import spsolve
from scikits.umfpack import spsolve, splu
import dune.ufl

```



```

import dune.fem
from dune.grid import cartesianDomain
from dune.alugrid import aluConformGrid as leafGrid
from dune.fem.function import integrate
from dune.fem.scheme import galerkin
from ufl import TrialFunction, TestFunction, inner, grad, dx, conditional, sin,
    pi, dot
from dune.ufl import Constant
from dune.fem.space import lagrange
import matplotlib.pyplot as plt
from dune.fem.plotting import plotPointData as plot
from ufl import as_vector
from scipy.sparse.linalg import spsolve as solver

# Create a grid on  $[0,1]^2$ 
domain = cartesianDomain([0,0],[1,1],[5,5])
gridView = leafGrid(domain)

space = lagrange(gridView, order=1, dimRange=2)

def initial(x):
    h = 0.01
    #T_val = 0.5 #50 * h # 0.5
    #T = Constant(T_val)
    g0 = lambda x, x0, T: conditional(
        x - x0 < -T/Constant(2),
        0,
        conditional(
            x - x0 > T/Constant(2),
            0,
            sin(2*pi/T*(x - x0))**3
        )
    )
    G = lambda x, y, x0, y0, T: g0(x, x0, T) * g0(y, y0, T)
    return 0.5 * G(x[0], x[1], 0.5, 0.5, 50*h)

# for the vector-valued function, we use initial(x) for u0 and set eta = 0
def initial_vector(x):
    return (initial(x), 0.0)

# interpolate the initial data into the space.
df = space.interpolate(initial_vector, name="solution")
# initialize discrete functions for the time loop.
u_prev = df.copy() # previous time step (u_prev holds [u0, eta])
u_h = space.interpolate([0]*2, name="u_h") # current solution

# trial and test function
u = TrialFunction(space)

```



```

v = TestFunction(space)

# time step and parameter epsilon
tau = Constant(1.0, name="tau") # timestep constant
eps = 0.05
eps2 = Constant(eps**2, name="eps2") # epsilon squared

#eyre splitting
eq1 = ((u[0] - u_prev[0]) / tau) * v[0] * dx \
      + dot(grad(u[1]), grad(v[0])) * dx

eq2 = u[1] * v[1] * dx \
      - (u[0]**3 - u_prev[0]) * v[1] * dx \
      - eps2 * dot(grad(u[0]), grad(v[1])) * dx

F = eq1 + eq2

equation = F == 0

# create the Galerkin scheme with Neumann (do-nothing) boundary conditions
scheme_obj = galerkin(equation, solver='gmres',
                      parameters={"linear.preconditioning.method": "jacobi",
                                   "nonlinear.linesearch": "simple"})

#time step
T_final = 2.0
t = 0.0
step=0
while t < T_final:
    print("Solving at time t =", t)
    info = scheme_obj.solve(target=u_h)
    u_prev.assign(u_h)
    t += float(tau.value)
    step += 1
    print(f"Step: {step}")

plot(u_h[0])
plt.show()

plot(u_h[1])

plt.show()

```

Task 10 - implicit

```

try:
    import dune.vem
except:
    print("This example needs 'dune.vem' - skipping")
    import sys

```

```

    sys.exit(0)
from scipy.sparse.linalg import spsolve
from scikits.umfpack import spsolve, splu
import dune.ufl
import dune.fem
from dune.grid import cartesianDomain
from dune.alugrid import aluConformGrid as leafGrid
from dune.fem.function import integrate
from dune.fem.scheme import galerkin
from ufl import TrialFunction, TestFunction, inner, grad, dx, conditional, sin,
    pi, dot
from dune.ufl import Constant
from dune.fem.space import lagrange
import matplotlib.pyplot as plt
from dune.fem.plotting import plotPointData as plot
from ufl import as_vector
from scipy.sparse.linalg import spsolve as solver

# create a grid on  $[0,1]^2$ 
domain = cartesianDomain([0,0],[1,1],[71,71])
gridView = leafGrid(domain)

space = lagrange(gridView, order=1, dimRange=2)

def initial(x):
    h = 0.01
    #T_val = 0.5 #50 * h # 0.5
    #T = Constant(T_val)
    g0 = lambda x, x0, T: conditional(
        x - x0 < -T/Constant(2),
        0,
        conditional(
            x - x0 > T/Constant(2),
            0,
            sin(2*pi/T*(x - x0))**3
        )
    )
    G = lambda x, y, x0, y0, T: g0(x, x0, T) * g0(y, y0, T)
    return 0.5 * G(x[0], x[1], 0.5, 0.5, 50*h)

# for the vector-valued function, we use initial(x) and eta
def initial_vector(x):
    u0 = initial(x)
    eta0 = u0**3 - u0
    return (u0, eta0)

# interpolate the initial data into the space.

```

```

df = space.interpolate(initial_vector, name="solution")
# initialize discrete functions for the time loop
u_prev = df.copy() # previous time step
u_h = space.interpolate([0]*2, name="u_h") # current solution

# trial and test function
u = TrialFunction(space)
v = TestFunction(space)

# time step and parameter epsilon
tau = Constant(1.0, name="tau") # timestep constant
eps = 0.05
eps2 = Constant(eps**2, name="eps2") # epsilon squared

eq1 = ((u[0] - u_prev[0]) / tau) * v[0] * dx \
      + dot(grad(u[1]), grad(v[0])) * dx

eq2 = (
    u[1] * v[1] * dx
    - ((u[0]**3) - u[0]) * v[1] * dx
    - eps2 * dot(grad(u[0]), grad(v[1])) * dx
)

F = eq1 + eq2

equation = F == 0

# create the Galerkin scheme with Neumann (do-nothing) boundary conditions:
scheme_obj = galerkin(equation, solver='gmres',
                      parameters={
                          "linear.verbose": True,
                          "linear.preconditioning.method": "jacobi"})

#time step
T_final = 2.0
t = 0.0
step=0
while t < T_final:
    print("Solving at time t =", t)
    info = scheme_obj.solve(target=u_h)
    u_prev.assign(u_h)
    t += float(tau.value)
    step += 1
    print(f"Step: {step}")

plot(u_h[0])

```

```
plt.show()

# Plot the eta component (the second component)
plot(u_h[1])

plt.show()
```